Authors:

- Cezary Suchorski 160310
- Bartosz Tarnachowicz 160304

Code repo: https://github.com/csuchorski/evolutionary_computation

# Hamiltonian cycle through a selected half of the nodes

**Evolutionary Computation · Laboratory 1**

This notebook is the report. The optimisation engine is written in C++ ( `src/main.cpp` ); it reads the instances, builds the distance matrix once and writes machine-readable results ( `results/*.csv` ). Everything shown below — tables, distributions, 2-D visualisations, best-tour listings and the discussion — is produced here in Python from those files.

| | |
|---|---|
| Instances | `TSPA.csv` , `TSPB.csv` — 200 nodes each |
| Selected nodes | $k = \lceil n/2 \rceil = 100$ |
| Solutions | 200 random, and 200 per greedy method (one start per node) |
| Seed | 12345 |

## 1. Problem description

Each instance describes **n = 200** points in the plane. Point $i$ has coordinates $(x_i, y_i)$ and an integer cost $c_i$. We must select exactly **k = ⌈n/2⌉ = 100** nodes and arrange them in one closed tour that visits every selected node exactly once, minimising

$$F(T) = \Sigma_{i=0}^{k-1} [\, c_{t_i} + d(t_i, t_{i+1}) \,], \quad t_k = t_0,$$

where d(i, j) is the Euclidean distance rounded to the nearest integer.

Both terms matter. A cheap node far from the current route may cost more in travel than it saves, and a geometrically convenient node may be expensive. Because $|T|$ is fixed, the node-cost term is an additive penalty of constant size, so the search is never pushed towards selecting fewer nodes.

## 2. Data and distance matrix

The files contain one row per node with the fields `x;y;cost` . The distance matrix is computed **once, immediately after reading**:

$$d(i, j) = \text{round}( \sqrt{( (x_i - x_j)^2 + (y_i - y_j)^2 )} )$$

with `std::llround` (half away from zero). The optimisation routines receive only `d[][]` and `cost[]` ; the coordinates are used by this notebook for plotting only.

The matrix is stored as **directed arcs** and no symmetry is assumed anywhere. Both supplied instances are Euclidean, hence d(i, j) = d(j, i), but the algorithms never rely on that property.

# 3. Algorithms

Each method constructs a sequence `T` of exactly `k` distinct nodes. Candidates are always ranked by their effect on the complete objective, never by distance alone.

**Notation.** `s` is the starting node, `cost[j]` is the cost of node `j`, `d[a][b]` is the distance from `a` to `b`, and `used[j]` says whether node `j` is already in `T`. The objective is evaluated only after construction:

$F(T) = \Sigma_{v \in T}\, \text{cost}[v] + \Sigma_{i=0}^{|T|-1}\, d(T[i], T[(i+1) \bmod |T|])$.

Ties are resolved deterministically by the smallest node index and then by the earliest insertion position.

## 3.1 Random solution

Choose `k` distinct nodes and visit them in a uniformly random order; there is no greedy scoring.

```
RANDOM(n, k):
    T ← k distinct nodes chosen uniformly from {0, …, n−1}
    SHUFFLE(T)
    return T
```

For the experiment this constructor is called 200 times.

## 3.2 Nearest neighbour, appending only at the end

The path has a single growing point: its last node. Each step adds the unused node with the smallest increase in the objective.

```
NN_APPEND(s, k):
    T ← [s]
    mark s as used
    while |T| < k:
        j* ← argmin ( cost[j] + d[T[last]][j] ) over unused j
        append j* to T
        mark j* as used
    return T
```

Only the forward arc `T[last] → j*` is added during construction. The closing arc `T[last] → T[0]` is part of `F(T)` and is added when the tour is evaluated.

## 3.3 Nearest neighbour, best of all positions

A new node may enter at the front, between two consecutive nodes, or at the end. For each unused node and each position, the exact change in the objective is:

```
INSERTION_DELTA(j, p):
    if p = 0:          return cost[j] + d[j][T[0]]
    if p = |T|:        return cost[j] + d[T[last]][j]
    a ← T[p−1];  b ← T[p]
    return cost[j] + d[a][j] + d[j][b] − d[a][b]
```

The constructor then takes the best `(node, position)` pair:

```
NN_ANY_POSITION(s, k):
    T ← [s]
    mark s as used
    while |T| < k:
        (j*, p*) ← argmin INSERTION_DELTA(j, p)
                    over unused j and p = 0, …, |T|
        insert j* at position p*
        mark j* as used
    return T
```

Inserting at the front adds only `j → T[0]`; appending adds only `T[last] → j`; an interior insertion replaces one arc `a → b` by `a → j` and `j → b`.

## 3.4 Greedy cycle

The cycle is closed after every insertion. A new node is placed into an existing edge `a → b`, so the old arc is removed and two new arcs are added.

```
CYCLE_DELTA(a, b, j):
    return cost[j] + d[a][j] + d[j][b] − d[a][b]

GREEDY_CYCLE(s, k):
    j* ← argmin ( cost[j] + d[s][j] + d[j][s] ) over j ≠ s
    T ← [s, j*]                          # two-node cycle s → j* → s
    mark s and j* as used
    while |T| < k:
        best ← +∞
        for each unused j:
            for each edge a → b of T:   # includes the wrap edge last → first
                Δ ← CYCLE_DELTA(a, b, j)
                if Δ < best:
                    best ← Δ;  (j*, a*) ← (j, a)
        insert j* immediately after a*
        mark j* as used
    return T
```

The wrap edge is treated like any other edge. Inserting at the end therefore costs

cost[j] + d[last][j] + d[j][first] − d[last][first],

not merely `cost[j] + d[last][j]`: the return arc is added and the previous closing arc is removed.

Because every method simply builds `T`, the same `EVALUATE(T)` routine computes the distance, node costs and objective for all 200 runs of every method.

```
Instance TSPA: n = 200, k = 100
  Random                       min 235605   max 292306   avg 261429.4    (0.4 ms)
  Nearest neighbour (append at end) min 84362    max 90075    avg 87476.6     (5.0 ms)
  Nearest neighbour (any position) min 72328    max 77022    avg 74347.0     (242.8 ms)
  Greedy cycle                 min 72590    max 75281    avg 73793.4     (359.6 ms)
Instance TSPB: n = 200, k = 100
  Random                       min 183300   max 229903   avg 207269.3    (0.3 ms)
  Nearest neighbour (append at end) min 56007    max 63166    avg 59397.3     (4.9 ms)
  Nearest neighbour (any position) min 52915    max 66164    avg 59475.5     (241.2 ms)
  Greedy cycle                 min 55655    max 63734    avg 59649.9     (355.7 ms)

Results written to results/
  summary.csv, runs.csv, best_tours.csv, best_tours_columns.csv

Python 3.14.8 · pandas 3.0.6 · matplotlib 3.11.2
```

| | Instance | File | n | k | Min cost | Max cost |
|---|---|---|---|---|---|---|
| **0** | TSPA | TSPA.csv | 200 | 100 | 0 | 1,997 |
| **1** | TSPB | TSPB.csv | 200 | 100 | 3 | 999 |

# 4. Computational experiment

For every instance, 200 random solutions were generated and each greedy method was run 200 times. With n = 200 and 200 runs, run *r* starts from node *r*, so every node is used exactly once as a starting point. The random generator is `std::mt19937` seeded with 12345, which makes the results reproducible. All runs are independent; no solution is improved after construction. The engine is compiled with `g++ -O2 -std=c++17`.

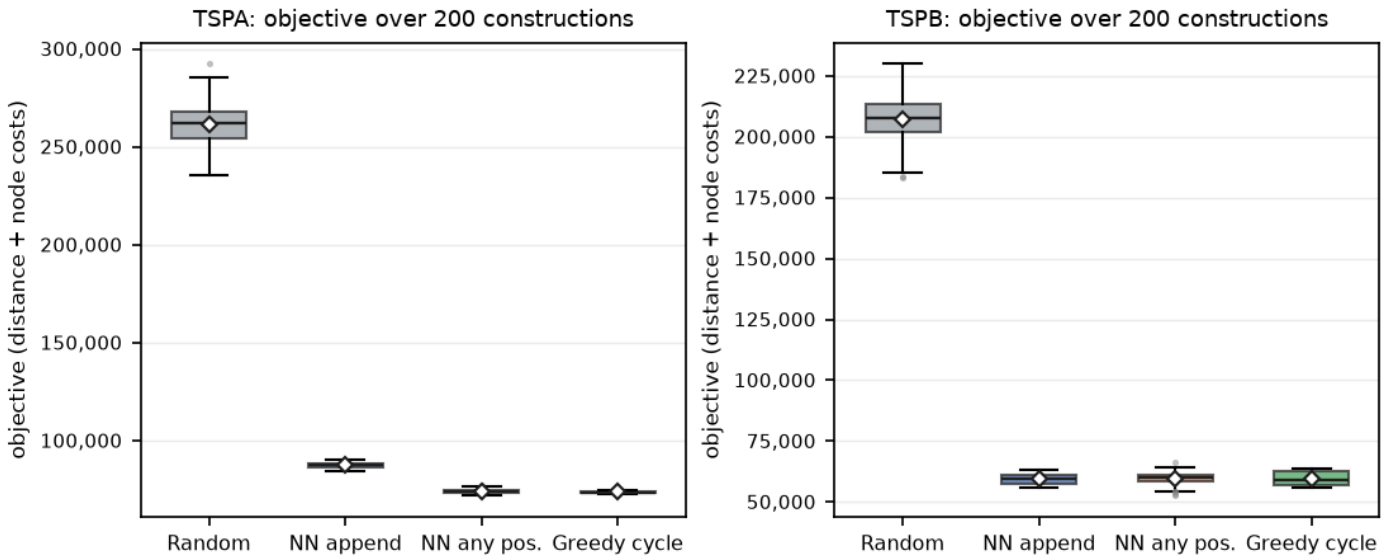
# 5. Results

The table gives the minimum, maximum and average objective over the 200 constructions, with the two instances side by side.

| **Instance** | **TSPA** | | | **TSPB** | | |
|---|---|---|---|---|---|---|
| **Statistic** | **Min** | **Max** | **Average** | **Min** | **Max** | **Average** |
| **Random** | 235,605 | 292,306 | 261,429.4 | 183,300 | 229,903 | 207,269.3 |
| **NN append** | 84,362 | 90,075 | 87,476.6 | 56,007 | 63,166 | 59,397.3 |
| **NN any pos.** | 72,328 | 77,022 | 74,347.0 | 52,915 | 66,164 | 59,475.5 |
| **Greedy cycle** | 72,590 | 75,281 | 73,793.4 | 55,655 | 63,734 | 59,649.9 |

*NN = nearest neighbour; "append" inserts only at the end of the path, "any pos." at the front, inside or at the end.*

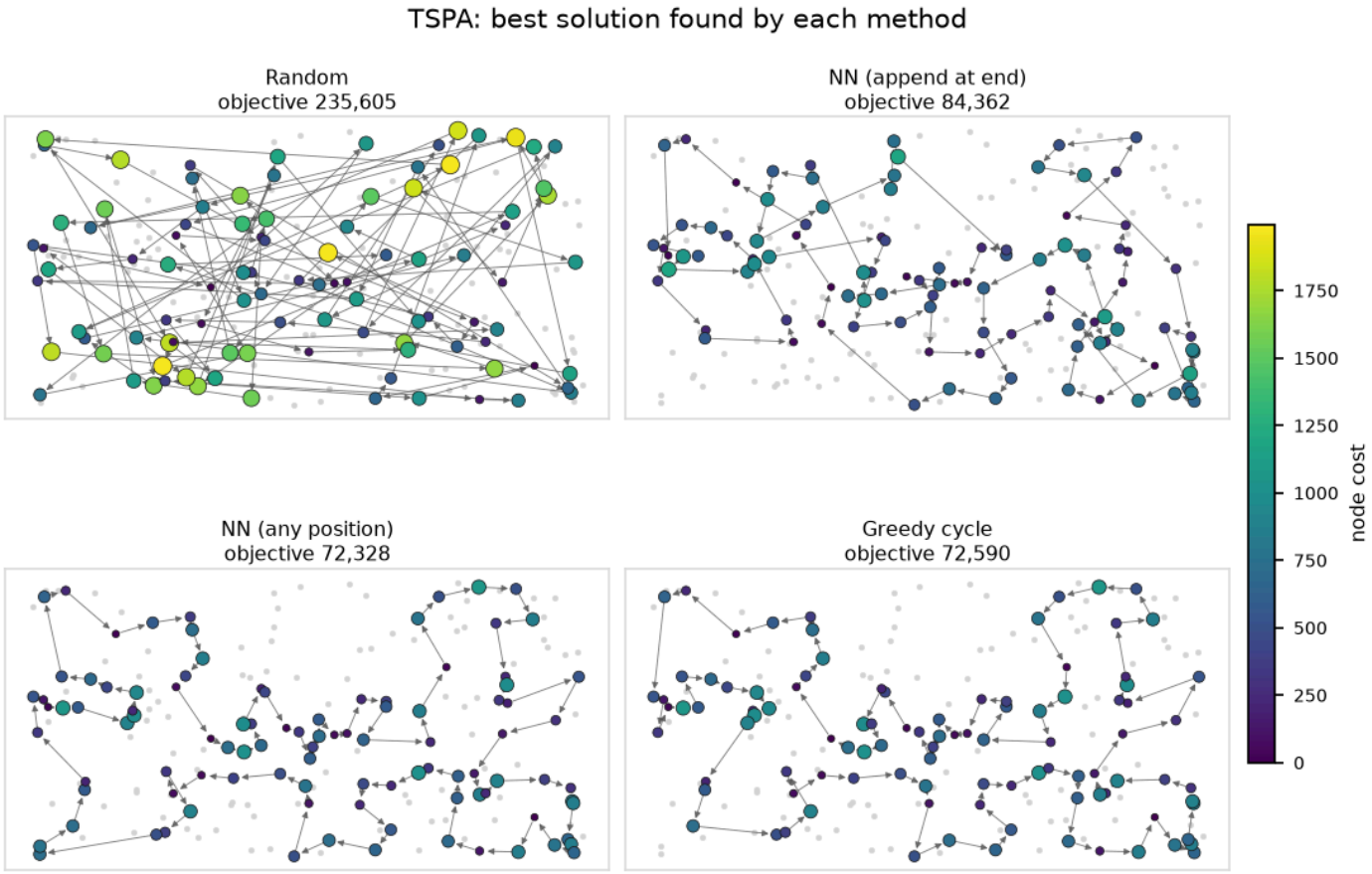
The minimum, maximum and average are the required statistics; the box plots below show the whole distribution of the 200 solutions per method. White diamonds mark the means, horizontal lines the medians.

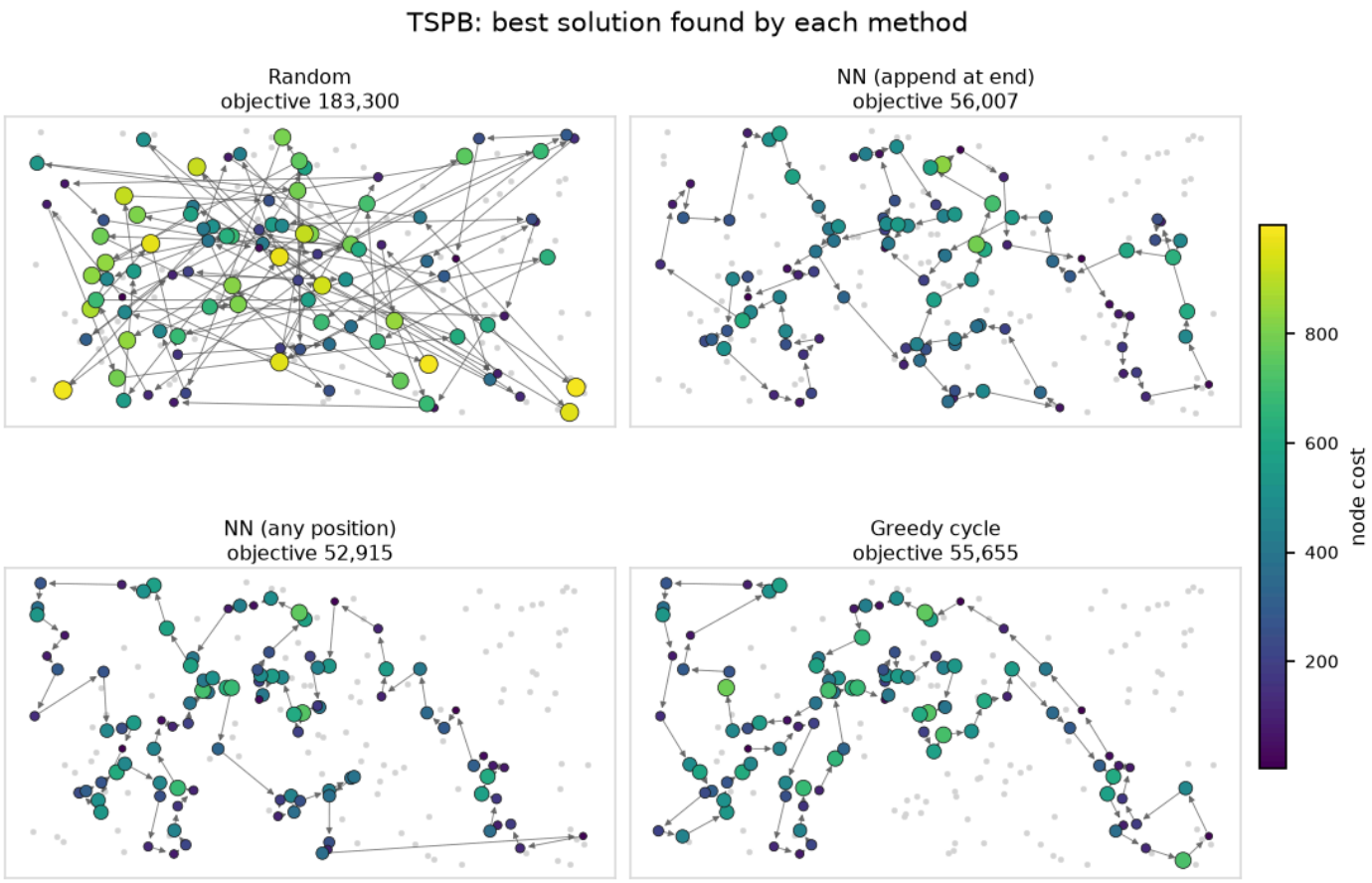



Random construction is far behind the greedy methods and has the widest spread. Among the greedy methods the differences are much smaller, so the choice of insertion rule matters more than the starting node.

## 6. Best solutions

Each panel shows the cycle built by one method. Grey dots are nodes that were **not** selected. Selected nodes are drawn with a colour **and** a radius that both encode the node cost (dark purple, small = cheap; yellow, large = expensive). Thin arrows show the direction of travel along the cycle.






## 7. Best solutions as node lists

All indices are **0-based**; the list is the cycle order and the closing arc runs from the last node back to the first one. For spreadsheet use, `results/best_tours_columns.csv` stores the same eight tours as columns: every column contains 100 node indices, one per row, so it can be copied directly into Excel without reformatting.

The best solutions have been checked with the Excel solution checker.

TSPA

Random — objective 235,605 (distance 143,298, costs 92,307), first node 40:
  40 113 74 96 71 102 31 137 51 22 92 66 171 32 127 194
  143 15 37 144 193 161 85 41 136 108 29 75 13 155 79 76
  89 45 30 10 149 180 154 185 35 133 129 70 132 16 106 151
  176 156 116 188 57 64 2 111 107 192 158 163 179 36 59 82
  120 124 162 54 183 5 72 55 84 197 196 157 153 186 99 168
  177 18 4 123 119 166 112 141 135 65 187 198 1 109 93 46
  17 95 195 199

NN (append at end) — objective 84,362 (distance 31,319, costs 53,043), first node 54:
  54 9 33 36 177 67 144 53 135 58 134 176 174 106 89 107
  57 182 151 93 50 3 1 164 178 29 136 186 184 47 159 130
  153 78 88 193 170 146 75 64 150 42 113 43 85 142 8 131
  84 158 192 181 114 141 180 132 76 80 48 31 63 149 168 97
  127 187 30 123 152 108 60 116 49 11 119 160 83 90 51 16
  35 87 45 140 52 179 185 28 102 156 157 71 91 38 100 4
  112 139 115 5

NN (any position) — objective 72,328 (distance 21,816, costs 50,512), first node 104:
  104 129 69 127 97 168 116 149 63 11 49 160 83 123 100 152
  108 60 31 48 80 91 157 71 4 76 132 180 28 102 185 35
  156 51 16 67 144 140 53 179 52 177 36 33 9 54 131 174
  8 164 186 136 163 29 101 178 1 169 3 50 147 47 93 106
  42 89 113 130 159 146 170 153 193 78 88 107 64 57 182 151
  150 176 72 134 84 58 158 192 181 114 135 73 45 87 141 187
  30 189 2 171

Greedy cycle — objective 72,590 (distance 21,570, costs 51,020), first node 31:
  31 116 149 168 63 11 49 60 108 152 100 90 123 83 160 97
  127 69 171 2 189 30 187 141 87 45 73 135 114 181 192 158
  58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170
  146 159 130 113 89 42 106 93 47 147 50 3 169 1 178 101
  29 163 136 186 164 142 8 174 131 54 9 33 36 177 52 179
  53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71
  157 91 80 48

TSPB

Random — objective 183,300 (distance 133,397, costs 49,903), first node 168:
  168 15 169 25 195 22 147 189 177 102 161 60 181 59 57 149
  158 184 75 138 163 160 192 23 172 62 26 123 120 52 171 134
  128 198 137 194 175 19 98 85 114 90 33 105 162 34 101 109
  113 108 183 135 8 196 141 143 31 79 117 170 28 91 150 151
  47 186 164 45 155 94 190 54 74 130 191 53 179 4 156 29
  140 27 2 21 69 20 76 119 39 115 49 30 121 100 104 13
  166 167 43 55

NN (append at end) — objective 56,007 (distance 24,632, costs 31,375), first node 32:
  32 89 57 76 54 127 129 90 153 20 14 2 110 118 56 114
  196 10 148 143 78 192 162 104 185 184 3 151 69 172 133 120
  96 121 198 48 72 46 86 117 160 136 8 21 154 140 47 170
  125 177 6 132 29 197 175 134 45 60 40 71 155 97 115 180
  65 28 149 145 158 116 147 42 50 130 30 183 38 64 85 59
  167 51 87 179 82 88 171 99 67 24 34 181 152 73 98 27
  135 75 79 156

NN (any position) — objective 52,915 (distance 20,068, costs 32,847), first node 110:
  110 2 14 20 74 153 182 52 90 129 127 80 54 192 162 104
  76 185 57 89 86 46 72 3 184 173 151 122 117 8 160 69
  172 136 133 120 156 43 48 96 198 121 130 191 50 42 147 116
  37 146 13 149 28 145 65 158 177 125 26 170 197 36 132 29
  6 180 115 155 71 175 134 45 157 60 40 141 97 154 140 47
  38 21 176 30 183 189 168 64 59 85 51 73 167 152 34 24
  181 179 87 82

Greedy cycle — objective 55,655 (distance 19,634, costs 36,021), first node 174:
  174 36 197 6 180 115 170 97 140 47 155 71 175 134 45 157
  60 141 40 33 64 21 189 168 69 160 172 136 133 120 8 117

```
72 46 86 184 3 173 122 151 98 111 27 135 76 185 162 192
80 52 182 20 74 2 93 110 118 14 153 90 129 127 54 104
57 89 156 43 48 96 198 121 63 130 191 183 30 176 38 154
26 125 177 138 158 145 149 147 42 116 37 146 13 28 65 178
103 29 132 1
```

# 8. Observations

- No method wins both instances on average. The lowest average objective in the experiment is produced by append-at-end on TSPB, while greedy cycle is best on TSPA; the margins over the next method are only 0.7% and 0.1%. Absolute objective values should not be compared between instances because their node-cost scales differ.
- TSPA is the clearest result. Greedy cycle beats NN any-position on 144/200 starts and beats append-at-end on 200/200. Its mean route is 3.3% shorter than NN any-position (22,304 vs 23,071), while its mean node cost is only 0.4% higher (51,489 vs 51,276). The gain therefore comes from route quality, not cheaper nodes. Greedy cycle also has the smallest relative spread on TSPA (0.7%).
- TSPB reverses the trade-off. Append-at-end is only 0.1% better than NN any-position and 0.4% better than greedy on average. It achieves this by accepting a 44–49% longer mean route (26,882 vs 18,678/18,015) in exchange for a 20–22% lower mean node cost (32,515 vs 40,798/41,635). The three methods win 65–68 of 200 starts, so the ranking is weak; TSPB is also much more start-sensitive (CV 3.5–4.7% vs 0.7–1.5% on TSPA).
- Best-of-200 is a different criterion: NN (any position) produces the best single solution on TSPA (72,328) and the same method also produces the best single TSPB tour (52,915), which is the global best. On TSPB it is 5.5% below append-at-end's best tour, so the broader positional search has the best upside but also the higher variance; it is preferable when one good solution matters, while greedy cycle on TSPA and append-at-end on TSPB are preferable for average reliability.
- The leading TSPA methods are already close to the cheapest-100 node-cost floor: greedy averages 51,489 against 49,272 (+4.5%), so further gains must come mainly from travel. On TSPB, append-at-end is 32,515 against 27,292 (+19.1%), while the insertion methods are +49–53% above the floor; the TSPB winner is the only method that stays relatively close to the cheap-node set.
- Runtime grows with the amount of search. Append-at-end needs about 0.025 ms per construction, NN any-position 1.21 ms, and greedy cycle 1.79 ms. Greedy is roughly 72x slower than append-at-end and 1.5x slower than NN any-position; that extra work is justified on TSPA (0.7% better average) but not on TSPB (0.4% worse).
- Random is a baseline rather than a contender: it is roughly 3.5x worse on average, and its node-cost term alone is much higher (97,312 vs 51,489 on TSPA; 51,829 vs 32,515 on TSPB), so it fails on both parts of the objective, not only route length.

**Discussion.** The two instances reward different heuristics because the balance between travel and node cost is different. On TSPA the leading methods already select similarly cheap node sets, so route length decides; greedy cycle's closed-edge insertion shortens the average route and wins on 144/200 starts, making it the most reliable TSPA method. On TSPB, append-at-end accepts a much longer route in exchange for a substantially lower node-cost bill; the saving almost exactly offsets the extra travel, leaving a margin of only 0.1–0.4%, and the per-start wins are split 65–68, so there is no robust winner.

NN any-position is the best single-shot method: it finds the best individual tour on both instances, including the global best on TSPB, but its higher variance makes it less reliable on average. Runtime

reinforces the instance-dependent choice: greedy costs about 1.8 ms per construction versus 1.2 ms for NN any-position and 0.025 ms for append-at-end. The extra computation is worthwhile on TSPA but not on TSPB. The practical recommendation is therefore greedy cycle for TSPA-like instances, append-at-end for TSPB-like instances where cheaper nodes are worth a longer route, and NN any-position when one high-quality solution matters more than average reliability.

# 9. Reproduction

```
# 1. Python environment (once)
python3 -m venv .venv
.venv/bin/pip install nbconvert ipykernel matplotlib pandas numpy jupyterlab

# 2. Engine
make

# 3. Execute this notebook, export HTML with report.css and print the PDF
.venv/bin/python tools/render_report.py
```

`run.sh` performs all of these steps and can also be used to rebuild everything from scratch; the notebook can be opened for interactive use with `.venv/bin/jupyter lab`. The engine writes `results/summary.csv`, `results/runs.csv`, `results/best_tours.csv` and `results/best_tours_columns.csv`; this notebook writes the figures to `report/figures/` and the exported report to `report/report.html` and `report/report.pdf`.